مستندات فنی پروژه‌ی سرخیابون

اپلیکیشن پرو مجازی لباس با هوش مصنوعی

مورد	مقدار
توسعه‌دهنده	داوود ایوازخانی
شروع مستندسازی	مهر ۱۴۰۵
وضعیت	زیرساخت سرور (Backend) ساخته و تست شد — مرحله‌ی بعد: اپلیکیشن موبایل

این فایل، دفترچه‌ی ساخت پروژه است. هر بخشی که ساخته می‌شود، همین‌جا با توضیح کامل ثبت می‌شود: چه چیزی ساخته شد، چرا این‌طور طراحی شد، چطور کار می‌کند، و چطور می‌شود آن را تست یا تغییر داد.

فهرست

1. نمای کلی و ترتیب ساخت
2. معماری کلی سیستم
3. قراردادهای فنی پروژه
4. دفترچه‌ی ساخت (بخش به بخش)
5. راه‌اندازی و اجرا
6. کارهای باقی‌مانده پیش از انتشار

---

۱. نمای کلی و ترتیب ساخت

اپلیکیشن سرخیابون به کاربر اجازه می‌دهد عکس خودش را بدهد، لباسی از فروشگاه انتخاب کند و با هوش مصنوعی ببیند آن لباس روی خودش چطور است؛ سپس همان‌جا خرید کند.

ترتیب ساخت عمداً از سرور شروع می‌شود، چون اپ موبایل بدون سرور هیچ کاری نمی‌تواند بکند:

ترتیب	بخش	وضعیت
۱	اسکلت پروژه و تنظیمات	✅
۲	احراز هویت با شماره موبایل (accounts)	✅
۳	کاتالوگ محصولات، بنر و آیکون تبلیغاتی (catalog)	✅

ترتیب	بخش	وضعیت
۴	موتور پرو مجازی (tryon)	✅ (با موتور آزمایشی؛ اتصال GPU آماده)
۵	سبد خرید، سفارش و پرداخت (orders)	✅
۶	تست‌ها و بررسی نهایی سرور	✅ ۱۱۴ تست + ۲۴ بررسی زنده
۷	اپلیکیشن موبایل React Native	مرحله‌ی بعد

---

۲. معماری کلی سیستم

```
اپلیکیشن موبایل (React Native)
        │  HTTPS + JWT
        ▼
سرور Django + Django REST Framework  ⟶  پایگاه داده PostgreSQL
        │
        │  ثبت درخواست پرو در صف
        ▼
صف پردازش (Celery + Redis)
        │
        ▼
لایه‌ی قابل‌تعویض موتور هوش مصنوعی  (TryOnProvider)
        ├── MockProvider       ← بدون GPU، برای توسعه و تست
        ├── RunPodProvider     ← مدل CatVTON روی RunPod Serverless
        └── (هر سرویس دیگری در آینده، بدون تغییر بقیه‌ی کد)
        │
        ▼
ذخیره‌ی تصویر خروجی ← برگرداندن نشانی تصویر به اپ
```

اصل کلیدی: اپ هرگز مستقیم به GPU وصل نمی‌شود. همه‌چیز از سرور عبور می‌کند تا امنیت، کنترل هزینه و امکان تعویض موتور حفظ شود.

---

۳. قراردادهای فنی پروژه

موضوع	تصمیم	دلیل
زبان و فریم‌ورک سرور	Python 3.11، Django 5.1، DRF	پایدار، پنل مدیریت آماده، سرعت توسعه
احراز هویت	شماره موبایل + کد یک‌بارمصرف، توکن JWT	رایج‌ترین روش ورود در ایران؛ JWT برای اپ موبایل مناسب است

موضوع	تصمیم	دلیل
پایگاه داده	PostgreSQL در سرور، SQLite برای توسعه و تست	بدون تغییر کد، فقط با متغیر محیطی
صف پردازش	Celery + Redis	پردازش تصویر کند است و نباید سرور را معطل کند
مستندات API	drf-spectacular (Swagger)	توسعه‌دهنده‌ی اپ همه‌ی endpointها را یک‌جا می‌بیند
تنظیمات حساس	فقط از طریق متغیر محیطی (`.env`)	هیچ رمز و کلیدی داخل کد نیست
زبان کد	نام‌ها و توضیحات کد انگلیسی؛ پیام‌های کاربر فارسی	استاندارد رایج؛ اپ کاملاً فارسی است

---

۴. دفترچه‌ی ساخت

> هر بخش پس از ساخته شدن، در ادامه ثبت می‌شود.

بخش ۱ — اسکلت پروژه و تنظیمات

هدف: ساختن پایه‌ای که همه‌ی بخش‌های بعدی روی آن سوار می‌شوند؛ طوری که همان کد، بدون تغییر، هم روی لپ‌تاپ توسعه‌دهنده و هم روی سرور اصلی اجرا شود.

ساختار پوشه‌ها

```
sarkhiaboon/
├── .env.example            ← فهرست کامل تنظیمات قابل تغییر (بدون مقادیر محرمانه)
├── docker-compose.yml      ← اجرای کل سیستم با یک دستور
├── docs/
│   └── DEVELOPMENT_LOG.md ← همین فایل
└── backend/
    ├── manage.py
    ├── requirements.txt    ← نسخه‌ی دقیق همه‌ی کتابخانه‌ها (قفل‌شده)
    ├── Dockerfile
    ├── config/
    │   ├── settings.py     ← همه‌ی تنظیمات، خوانده‌شده از متغیرهای محیطی
    │   ├── urls.py         ← مسیرهای اصلی API
    │   ├── celery.py       ← راه‌اندازی صف پردازش
    │   └── wsgi.py
    └── apps/
        ├── core/           ← ابزارهای مشترک
        ├── accounts/       ← کاربران و ورود
        ├── catalog/        ← محصولات، دسته‌بندی، بنر
        ├── tryon/          ← موتور پرو مجازی
        └── orders/         ← سبد خرید، سفارش، پرداخت
```

تصمیم‌های مهم این بخش

۱. همه‌ی تنظیمات از متغیر محیطی خوانده می‌شود. هیچ رمز، کلید API یا نشانی سروری داخل کد نیست. فایل `.env.example` فهرست کامل را دارد. اگر برنامه در حالت عملیاتی (`DJANGO_DEBUG=false`) بدون کلید امنیتی اجرا شود، عمداً اجرا نمی‌شود تا اشتباهاً با کلید ناامن بالا نیاید.

۲. پایگاه داده‌ی دوحالته. اگر `POSTGRES_DB` تعریف شده باشد از PostgreSQL استفاده می‌شود؛ در غیر این صورت از SQLite. یعنی توسعه‌دهنده بدون نصب هیچ سرویسی می‌تواند پروژه را اجرا و تست کند.

۳. مسیرهای API نسخه‌دار هستند (`/api/v1/...`). وقتی اپ منتشر شد، نسخه‌های قدیمی اپ روی گوشی کاربران باقی می‌مانند. اگر روزی API تغییر اساسی کند، نسخه‌ی `v2` ساخته می‌شود و نسخه‌های قدیمی اپ خراب نمی‌شوند.

۴. زمان‌بندی خودکار کارهای دوره‌ای (Celery Beat). سه کار به‌صورت خودکار تکرار می‌شوند:

کار	تناوب	هدف
حذف عکس‌های شخصی منقضی	هر ۳۰ دقیقه	حریم خصوصی کاربر
علامت‌گذاری درخواست‌های پرو گیرکرده	هر ۵ دقیقه	کاربر بی‌نهایت منتظر نماند
آزادسازی موجودی سفارش‌های پرداخت‌نشده	هر ۱۰ دقیقه	کالا بی‌دلیل رزرو نماند

مسیرهای عمومی

مسیر	کاربرد
`/api/v1/`	همه‌ی endpointهای اپ
`/api/docs/`	مستندات تعاملی API (Swagger) برای توسعه‌دهنده‌ی اپ
`/admin/`	پنل مدیریت کارفرما (فارسی)
`/health/`	بررسی سلامت سرور برای مانیتورینگ

اجرای کل سیستم

```
cp .env.example .env        # سپس مقادیر را ویرایش کنید
docker compose up --build
```

این دستور پنج سرویس را بالا می‌آورد: API، کارگر صف (worker)، زمان‌بند (beat)، PostgreSQL و Redis.

> **وضعیت:** ✅ ساخته شد. (فایل Docker نوشته شده ولی در محیط ساخت، Docker برای اجرا در دسترس نبود؛ اجرای آن در اولین استقرار روی سرور بررسی می‌شود.)

بخش ۲ — احراز هویت با شماره موبایل (`apps/accounts`)

هدف: ورود بدون رمز عبور، فقط با شماره موبایل و کد پیامکی؛ به‌علاوه‌ی امکان استفاده‌ی «مهمان» برای کسی که می‌خواهد قبل از ثبت‌نام، اپ و پرو مجازی را امتحان کند.

فایل‌ها

فایل	نقش
`models.py`	مدل کاربر (`User`) و کد یک‌بارمصرف (`OTPCode`)
`phone.py`	یکسان‌سازی و اعتبارسنجی شماره موبایل ایرانی
`sms.py`	ارسال پیامک با قابلیت تعویض سرویس‌دهنده
`services.py`	منطق اصلی ورود: ارسال کد، بررسی کد، ساخت یا ادغام حساب
`signals.py`	اعلام ادغام حساب مهمان به بقیه‌ی بخش‌ها
`serializers.py / views.py / urls.py`	لایه‌ی API
`admin.py`	نمایش کاربران و کدها در پنل مدیریت

فایل	نقش
`tests.py`	۱۷ تست خودکار

مدل کاربر

- **ورود با شماره موبایل** به‌جای نام کاربری. مشتری هیچ‌وقت رمز عبور تعریف نمی‌کند.
- **کاربران پنل مدیریت** (کارفرما و کارمندان) با شماره موبایل + رمز عبور وارد پنل می‌شوند.
- **کاربر مهمان** شماره ندارد و فقط با یک توکن شناخته می‌شود.

جریان ورود (دو مرحله)

```
۱. POST /api/v1/auth/otp/request/   { "phone": "09121234567" }
   ← { "detail": "کد ورود ارسال شد.", "expires_in": 120 }

۲. POST /api/v1/auth/otp/verify/    { "phone": "09121234567", "code": "48213" }
   ← { "access": "...", "refresh": "...", "is_new_user": true, "user": {...} }
```

اپ توکن `access` را در سربرگ همه‌ی درخواست‌ها می‌فرستد: `Authorization: Bearer <access>`. توکن `access` یک ساعت و `refresh` سی روز اعتبار دارد. وقتی `access` منقضی شد، اپ با `POST /api/v1/auth/token/refresh/` توکن جدید می‌گیرد و کاربر لازم نیست دوباره کد بگیرد.

فیلد `is_new_user` به اپ می‌گوید آیا باید صفحه‌ی تکمیل پروفایل (نام) را نشان دهد یا نه.

جریان مهمان

```
POST /api/v1/auth/guest/   ← توکن برای یک حساب مهمان
```

اگر مهمان بعداً شماره‌اش را تأیید کند (با همان توکن مهمان در سربرگ):

وضعیت	نتیجه
شماره قبلاً ثبت نشده	همان حساب مهمان به حساب عادی تبدیل می‌شود؛ هیچ داده‌ای از دست نمی‌رود
شماره قبلاً حساب داشته	داده‌های مهمان (پروها، علاقه‌مندی‌ها، سبد خرید) به حساب قبلی منتقل و حساب مهمان غیرفعال می‌شود

انتقال داده‌ها با یک «سیگنال» (`guest_merged`) انجام می‌شود. هر بخش دیگر پروژه خودش مسئول انتقال داده‌های خودش است؛ بخش حساب کاربری لازم نیست از جزئیات بقیه خبر داشته باشد.

امنیت

سازوکار	توضیح
ذخیره‌ی درهم‌شده‌ی کد	خود کد در پایگاه داده ذخیره نمی‌شود، فقط HMAC آن. اگر پایگاه داده نشت کند، کدها قابل استفاده نیستند
عمر کوتاه کد	پیش‌فرض ۱۲۰ ثانیه
فاصله‌ی ارسال مجدد	پیش‌فرض ۶۰ ثانیه برای هر شماره (جلوگیری از هدر رفتن هزینه‌ی پیامک)
سقف تلاش	پس از ۵ تلاش اشتباه، کد باطل می‌شود
یک‌بارمصرف	هر کد فقط یک بار قابل استفاده است
محدودیت نرخ درخواست	۵ درخواست کد در ساعت برای هر IP، ۲۰ تلاش تأیید، ۱۰ ساخت مهمان
توکن نامعتبر مانع ورود نیست	اگر اپ توکن منقضی‌شده‌ای در حافظه داشته باشد، ورود دوباره‌ی کاربر مسدود نمی‌شود

یکسان‌سازی شماره

همه‌ی این ورودی‌ها به `09121234567` تبدیل می‌شوند، تا یک نفر با دو شکل نوشتن شماره‌اش دو حساب جدا نسازد:

`0912-123-4567` · `0912 123 4567` · `9121234567` · `00989121234567` · `+989121234567` · `۰۹۱۲۱۲۳۴۵۶۷`

ارسال پیامک

سرویس‌دهنده‌ی پیامک با یک خط در `.env` عوض می‌شود:

مقدار `SMS_BACKEND`	کاربرد
`apps.accounts.sms.ConsoleSMSBackend`	توسعه: کد در لاگ سرور چاپ می‌شود
`apps.accounts.sms.KavenegarSMSBackend`	عملیاتی: ارسال با کاوه‌نگار (متد `verify/lookup`)
`apps.accounts.sms.LocMemSMSBackend`	فقط برای تست‌های خودکار

برای کاوه‌نگار باید در پنل آن یک قالب پیامک تأیید ساخته شود که شامل `%token` باشد و نامش در `KAVENEGAR_OTP_TEMPLATE` قرار گیرد. افزودن سرویس دیگر (مثلاً ملی‌پیامک) فقط یک کلاس جدید در `sms.py` است.

اگر ارسال پیامک شکست بخورد، کد ساخته‌شده حذف می‌شود تا کاربر بلافاصله بتواند دوباره درخواست کند (پاسخ `503`). اگر زودتر از ۶۰ ثانیه دوباره درخواست کند، پاسخ `429` با پیام «لطفاً N ثانیه‌ی دیگر...» می‌گیرد.

فهرست endpointها

متد	مسیر	ورود لازم	کاربرد
POST	`/api/v1/auth/otp/request/`	خیر	ارسال کد

متد	مسیر	ورود لازم	کاربرد
POST	`/api/v1/auth/otp/verify/`	خیر (اختیاری برای مهمان)	تأیید کد و دریافت توکن
POST	`/api/v1/auth/guest/`	خیر	ساخت حساب مهمان
POST	`/api/v1/auth/token/refresh/`	خیر	تمدید توکن
GET / PATCH	`/api/v1/auth/me/`	بله	مشاهده و ویرایش نام

تست‌ها

۱۷ تست خودکار، همه موفق. موارد پوشش‌داده‌شده: همه‌ی شکل‌های شماره، ورود کامل، کاربر تکراری، فاصله‌ی ارسال مجدد، قفل پس از تلاش‌های اشتباه، عدم استفاده‌ی مجدد از کد، کد منقضی، ارقام فارسی، خطای سرویس پیامک، ارتقای مهمان، ادغام مهمان با حساب موجود، توکن نامعتبر، ویرایش پروفایل، و عدم امکان تغییر شماره از طریق ویرایش پروفایل.

```
cd backend && DJANGO_DEBUG=true python manage.py test apps.accounts
```

> **وضعیت:** ✅ ساخته و تست شد.

بخش ۳ — کاتالوگ محصولات، بنر و آیکون تبلیغاتی (`apps/catalog`)

هدف: همه‌ی چیزهایی که کاربر در اپ می‌بیند و مرور می‌کند: دسته‌بندی‌ها، محصولات با سایز و رنگ و موجودی، بنرهای صفحه‌ی اصلی، آیکون‌های تبلیغاتی (درخواست کارفرما)، و علاقه‌مندی‌ها.

مدل‌های داده

مدل	نقش	نکات مهم
`Category`	دسته‌بندی درختی (مثلاً زنانه › بلوز)	آیکون، «نمایش در صفحه‌ی اصلی»، و **نوع پرو مجازی پیش‌فرض**
`Product`	محصول	قیمت به تومان، قیمت قبل از تخفیف، تصویر اصلی، **تصویر مخصوص پرو**، فعال/غیرفعال بودن پرو
`ProductVariant`	ترکیب سایز/رنگ قابل خرید	موجودی جداگانه، کد انبار (SKU)، قیمت اختصاصی اختیاری
`ProductImage`	گالری تصاویر محصول	ترتیب‌پذیر
`Banner`	بنر تبلیغاتی	دو جایگاه (اسلایدر بالا / میانه‌ی صفحه)، زمان شروع و پایان
`HomeShortcut`	آیکون تبلیغاتی صفحه‌ی اصلی	زمان شروع و پایان، لینک به هر جا

مدل	نقش	نکات مهم
`Favorite`	علاقه‌مندی کاربر	هر محصول فقط یک بار برای هر کاربر

نوع پرو مجازی

هر محصول یکی از این انواع را دارد. این مقدار مستقیماً به موتور هوش مصنوعی فرستاده می‌شود تا بداند کدام بخش بدن را بپوشاند (منطبق با دسته‌بندی‌های مدل CatVTON):

مقدار	معنا
`upper_body`	بالاتنه (بلوز، پیراهن، تیشرت، کت)
`lower_body`	پایین‌تنه (شلوار، دامن)
`dresses`	لباس یک‌تکه (مانتو بلند، پیراهن زنانه)
`shoes`	کفش — **فعلاً پیاده‌سازی نشده**؛ منتظر تصمیم کارفرما و تست امکان‌سنجی
`none`	بدون پرو (مثلاً کیف، اکسسوری)

وراثت از دسته‌بندی: کافی است نوع پرو یک بار روی دسته‌ی «شلوار» تنظیم شود؛ همه‌ی محصولات آن دسته خودکار `lower_body` می‌شوند. در صورت نیاز، هر محصول می‌تواند نوع خودش را جداگانه داشته باشد.

تصویر مخصوص پرو: مدل هوش مصنوعی با عکس تمیز خود لباس (بدون مدل انسانی، پس‌زمینه‌ی ساده) بهترین نتیجه را می‌دهد. برای هر محصول می‌شود این عکس را جدا بارگذاری کرد؛ اگر خالی بماند، تصویر اصلی محصول استفاده می‌شود. **این نکته باید به کارفرما گفته شود:** کیفیت عکس لباس، مستقیماً کیفیت پرو را تعیین می‌کند.

بنرها و آیکون‌های تبلیغاتی (درخواست کارفرما)

هر دو از پنل مدیریت کنترل می‌شوند و این امکانات را دارند:

- **زمان‌بندی:** با تعیین «شروع نمایش» و «پایان نمایش»، بنر در زمان مقرر خودش ظاهر و ناپدید می‌شود. مثلاً بنر حراج یلدا را از قبل می‌سازید و خودکار در شب یلدا نمایش داده می‌شود.
- **مقصد لینک:** با زدن روی بنر یا آیکون، کاربر به یک محصول، یک دسته‌بندی، یا یک نشانی اینترنتی می‌رود. API مقصد را همیشه در یک قالب ثابت برمی‌گرداند تا کار اپ ساده باشد:

```
"link": { "type": "product",  "slug": "بلوز-حریر" }
"link": { "type": "category", "slug": "شلوار" }
"link": { "type": "url",      "url": "https://..." }
"link": { "type": "none" }
```

- **اعتبارسنجی:** اگر نوع لینک «محصول» انتخاب شود ولی محصولی تعیین نشود، پنل اجازه‌ی ذخیره نمی‌دهد.

صفحه‌ی اصلی در یک درخواست

`GET /api/v1/catalog/home/` همه‌ی محتوای صفحه‌ی اصلی اپ را یک‌جا برمی‌گرداند:

کلید	محتوا
`banners_top`	اسلایدر بالای صفحه
`shortcuts`	ردیف آیکون‌های تبلیغاتی
`categories`	دسته‌بندی‌هایی که «نمایش در صفحه‌ی اصلی» دارند (با آیکون)
`banners_middle`	بنرهای میانه‌ی صفحه
`new_products`	۱۰ محصول جدید
`discounted_products`	۱۰ محصول تخفیف‌دار
`tryon_products`	۱۰ محصول قابل پرو مجازی

دلیل: اگر اپ برای صفحه‌ی اصلی ۷ درخواست جدا بفرستد، روی اینترنت ضعیف موبایل صفحه کند و تکه‌تکه بار می‌شود. یک درخواست یعنی صفحه‌ی اصلی سریع و یکجا ظاهر می‌شود.

فیلترهای فهرست محصولات

`GET /api/v1/catalog/products/?...`

پارامتر	مثال	توضیح
`category`	`زنانه`	شامل همه‌ی زیردسته‌ها
`min_price / max_price`	`max_price=600000`	بازه‌ی قیمت (تومان)
`size` / `color`	`size=M`	حساس به حروف کوچک و بزرگ نیست
`tryon`	`true`	فقط محصولات قابل پرو
`in_stock`	`true`	فقط موجودها
`discounted`	`true`	فقط تخفیف‌دارها
`search`	`جین`	جست‌وجو در نام، توضیحات و نام دسته
`ordering`	`price` یا `-price` یا `-created_at`	مرتب‌سازی
`page` / `page_size`		صفحه‌بندی (پیش‌فرض ۲۰، حداکثر ۱۰۰)

تصمیم‌های فنی

- **نامک (slug) فارسی:** نشانی محصول «بلوز-حریر» است، نه «product-123». برای اشتراک‌گذاری لینک و سئوی نسخه‌ی وب آینده خواناتر است. اگر نام تکراری باشد، خودکار `-2` ، `-3` و… اضافه می‌شود.

- **نشانی کامل تصاویر:** همه‌ی تصاویر با نشانی کامل (`https://...`) برگردانده می‌شوند تا اپ مستقیم بارشان کند. در سرور اصلی با `PUBLIC_BASE_URL` تنظیم می‌شود (برای وقتی که پشت CDN یا پروکسی هستیم).
- **مرور بدون ورود:** کاتالوگ برای همه باز است. توکن منقضی‌شده هم مانع مرور نمی‌شود.
- **پرچم علاقه‌مندی:** هر محصول در فهرست فیلد `is_favorite` دارد تا اپ آیکون قلب را درست نشان دهد، بدون درخواست جداگانه برای هر محصول.
- **بهینه‌سازی پایگاه داده:** تنوع‌ها و تصاویر با `prefetch_related` یکجا خوانده می‌شوند، نه یکی‌یکی.
- **حذف ناخواسته ممنوع:** دسته‌بندی‌ای که محصول دارد قابل حذف نیست (`PROTECT`)، تا کالاها بی‌دسته نمانند.
- **ادغام مهمان:** علاقه‌مندی‌های مهمان هنگام ورود به حساب موجود منتقل می‌شوند (بدون تکرار).

پنل مدیریت

- فهرست محصولات با تصویر کوچک، موجودی کل، و امکان ویرایش سریع قیمت و وضعیت از همان فهرست
- افزودن سایز/رنگ و گالری تصاویر داخل صفحه‌ی خود محصول
- دسته‌بندی‌ها با پیش‌نمایش آیکون و تنظیم سریع «نمایش در صفحه‌ی اصلی» و ترتیب
- بنرها و آیکون‌های تبلیغاتی با پیش‌نمایش تصویر و بخش جداگانه‌ی «زمان‌بندی نمایش»

فهرست endpointها

متد	مسیر	ورود لازم
GET	`/api/v1/catalog/home/`	خیر
GET	`/api/v1/catalog/categories/`	خیر
GET	`/api/v1/catalog/products/`	خیر
GET	`/api/v1/catalog/products/<slug>/`	خیر
GET	`/api/v1/catalog/banners/?position=`	خیر
GET / POST	`/api/v1/catalog/favorites/`	بله
DELETE	`/api/v1/catalog/favorites/<product_id>/`	بله

موارد عمداً ساخته‌نشده

- **دسته‌بندی شهر و منطقه:** منتظر تصمیم کارفرما (گزینه‌ی الف یا ب نقشه‌ی راه). ساختار فعلی طوری است که افزودن آن فقط یک مدل و یک فیلتر جدید است و بخش‌های موجود تغییر نمی‌کنند.

تست‌ها

۲۳ تست خودکار، همه موفق: ساخت نامک فارسی و جلوگیری از تکرار، وراثت نوع پرو، اعتبارسنجی تخفیف، جلوگیری از حلقه در درخت دسته‌بندی، اعتبارسنجی لینک بنر، همه‌ی فیلترها، جست‌وجو و مرتب‌سازی، پنهان بودن محصولات غیرفعال،

درخت دسته‌بندی، نمایش فقط بنرهای در بازه‌ی زمانی، محتوای کامل صفحه‌ی اصلی، علاقه‌مندی (افزودن تکراری، حذف، پرچم)، انتقال علاقه‌مندی‌های مهمان، و عدم نمایش کفش به‌عنوان «قابل پرو» تا وقتی موتور آن را پشتیبانی نکند (بخش ۶).

> **وضعیت:** ✅ ساخته و تست شد.

بخش ۴ — موتور پرو مجازی (`apps/tryon`)

هدف: قلب محصول. کاربر عکسش را می‌دهد، لباس را انتخاب می‌کند و تصویر پوشیدن آن لباس را می‌گیرد؛ با سه شرط: **موتور هوش مصنوعی قابل تعویض باشد، هزینه‌ی GPU کنترل شود، و حریم خصوصی کاربر حفظ شود.**

فایل‌ها

فایل	نقش
`models.py`	عکس کاربر (`PersonPhoto`) و درخواست پرو (`TryOnJob`)
`imaging.py`	بررسی کیفیت عکس، حذف اطلاعات مکانی، کوچک‌سازی، واترمارک
`providers/base.py`	**قرارداد** هر موتور هوش مصنوعی
`providers/mock.py`	موتور آزمایشی بدون GPU، برای توسعه و تست
`providers/runpod.py`	اتصال به RunPod Serverless (مثلاً CatVTON)
`services.py`	منطق اصلی: بارگذاری عکس، سهمیه، حافظه‌ی نتایج، ساخت درخواست
`tasks.py`	کارهای پس‌زمینه: پردازش پرو، حذف عکس‌های منقضی، رسیدگی به درخواست‌های گیرکرده
`signals.py`	حذف فایل‌ها هنگام حذف رکورد، انتقال داده‌ی مهمان
`admin.py`	گزارش در پنل مدیریت
`tests.py`	۳۹ تست خودکار

جریان کامل از دید اپ

```
۱. POST /api/v1/tryon/photos/            (multipart، فیلد image)
   ← { "id": "<photo_id>", "image": "...", "expires_at": "..." }
   (اگر عکس مناسب نباشد: 400 با دلیل فارسی (مثلاً «عکس تار است…»)

۲. POST /api/v1/tryon/jobs/              { "product_id": 12, "photo_id": "<photo_id>" }
   ← 202 { "id": "<job_id>", "status": "queued", "queue_position": 3, ... }
   (یا 200 با نتیجه‌ی آماده، اگر همین پرو قبلاً انجام شده باشد)

۳. GET /api/v1/tryon/jobs/<job_id>/      هر ۲ ثانیه، تا وقتی status برابر completed یا failed شود
   ← { "status": "completed", "result_image": "https://...", "generation_seconds": 7.8,
... }
```

کاربر عکسش را **یک بار** بارگذاری می‌کند و با همان عکس هر تعداد لباس را امتحان می‌کند (ایده‌ی «کمد لباس مجازی» در پروپوزال).

وضعیت‌ها و نمایش در اپ

این همان نمایش مرحله‌ای است که کارفرما خواسته بود. هر وضعیت یک برچسب فارسی آماده (`status_label`) دارد:

`status`	`status_label`	نمایش پیشنهادی در اپ
—	—	«در حال آماده‌سازی» (هنگام بارگذاری عکس، سمت اپ)
`queued`	در صف پردازش	«در صف پردازش — نفر N» (از `queue_position`)
`processing`	در حال پرو لباس	انیمیشن در حال پرو
`completed`	آماده شد	نمایش تصویر + «آماده شد (۷٫۸ ثانیه)» از `generation_seconds`
`failed`	ناموفق	نمایش `error_message` (همیشه یک جمله‌ی فارسی قابل فهم)

معماری قابل‌تعویض موتور — مهم‌ترین تصمیم فنی پروژه

همه‌ی پروژه فقط با یک «قرارداد» حرف می‌زند (`TryOnProvider`): «دو عکس و نوع لباس بگیر، یک تصویر برگردان». هر موتوری که این قرارداد را رعایت کند، قابل استفاده است.

```
TRYON_PROVIDER=apps.tryon.providers.mock.MockProvider      ← الان (توسعه)
TRYON_PROVIDER=apps.tryon.providers.runpod.RunPodProvider  ← بعد از آماده شدن GPU
```

تعویض موتور = تغییر یک خط در `.env`. هیچ بخش دیگری از سرور و هیچ‌چیز در اپ موبایل تغییر نمی‌کند. این یعنی هر دو مسیر پیشنهادی به کارفرما (شروع با سرویس آماده یا RunPod از ابتدا) با همین کد پوشش داده می‌شود. اگر فردا سرویس دیگری بهتر یا ارزان‌تر شد، یا RunPod از ایران در دسترس نبود، فقط یک کلاس جدید نوشته می‌شود.

موتور آزمایشی (`MockProvider`): تصویر لباس را روی ناحیه‌ی بدن عکس می‌چسباند. **پرو واقعی نیست** و فقط برای این است که کل مسیر (اپ، صف، سرور، ذخیره، نمایش) بدون GPU ساخته و تست شود. نباید در نسخه‌ی عملیاتی استفاده شود.

اتصال به RunPod

`RunPodProvider` از روش «ارسال و پیگیری» (`/run` سپس `/status`) استفاده می‌کند، نه روش انتظار مستقیم (`/runsync`)، چون راه‌اندازی اولیه‌ی سرد (cold start) ممکن است طولانی‌تر از مهلت روش مستقیم باشد.

قرارداد با کارگر GPU — هر ایمیج Docker که روی RunPod قرار می‌گیرد باید دقیقاً این قالب را رعایت کند:

```
ورودی:  { "input": { "person_image": "<base64 JPEG>", "garment_image": "<base64>",
                     "category": "upper_body | lower_body | dresses", "job_id": "
<uuid>" } }
خروجی موفق:  { "image": "<base64>", "format": "png" }
خروجی خطا:   { "error": "no person detected" }
```

رفتار در برابر خطا:

وضعیت	نوع	نتیجه
قطع شبکه، خطای ۴۲۹، ۵xx (سرویس شلوغ)	موقت	تلاش مجدد خودکار
کار روی RunPod با وضعیت FAILED یا TIMED_OUT	موقت	تلاش مجدد خودکار
گذشتن از مهلت (پیش‌فرض ۱۸۰ ثانیه)	موقت	کار روی RunPod **لغو** می‌شود (تا هزینه ادامه پیدا نکند) و دوباره تلاش می‌شود
کلید API نامعتبر (۴۰۱/۴۰۳)	دائمی	بدون تلاش مجدد
کارگر اعلام کند ورودی ایراد دارد (مثلاً آدمی در عکس نیست)	دائمی	بدون تلاش مجدد، پیام «لطفاً عکس تمام‌قد بگیرید»

اندازه‌گیری واقعی cold start: RunPod دو زمان گزارش می‌دهد: `delayTime` (انتظار در صف + راه‌اندازی سرد) و `executionTime` (پردازش خالص). هر دو روی هر درخواست ذخیره می‌شوند (`provider_meta`). بعد از لانچ، به‌جای حدس زدن، با داده‌ی واقعی مشخص می‌شود cold start چقدر زمان و هزینه اضافه می‌کند — همان موضوعی که در پروپوزال فنی به کارفرما گفته شد.

> **وضعیت RunPod:** کد اتصال کامل است و با شبیه‌سازی پاسخ‌های RunPod تست شده (۸ تست)، اما هنوز به یک endpoint واقعی متصل نشده، چون ساخت ایمیج Docker مدل CatVTON و خود حساب RunPod هنوز وجود ندارد. ساخت آن ایمیج همان «کار اضافه»ی مسیر دوم در پروپوزال فنی است.

کنترل هزینه‌ی GPU

سازوکار	توضیح
حافظه‌ی نتایج (cache)	اگر همان عکس با همان لباس دوباره پرو شود، نتیجه‌ی قبلی فوراً برگردانده می‌شود؛ **بدون هیچ فراخوانی GPU**. کلید حافظه از اثرانگشت عکس + اثرانگشت تصویر لباس + نوع لباس + نام موتور ساخته می‌شود. اگر تصویر لباس در پنل عوض شود، خودکار نتیجه‌ی جدید ساخته می‌شود
رد عکس بد قبل از ارسال	عکس کوچک، افقی، تاریک، پرنور یا تار، قبل از رسیدن به GPU رد می‌شود. هر پردازش بی‌نتیجه یعنی هزینه‌ی هدررفته
سهمیه‌ی روزانه	مهمان: ۳ پرو در روز (سپس پیام «با شماره موبایل وارد شوید» — ابزار جذب ثبت‌نام). کاربر: ۳۰ پرو در روز. هر دو از `.env` قابل تغییرند
کوچک‌سازی عکس	عکس‌های بزرگ‌تر از ۱۵۳۶ پیکسل کوچک می‌شوند (زمان پردازش GPU و حجم ارسال کمتر)
لغو کار منقضی‌شده	اگر کاری روی RunPod از مهلت بگذرد، لغو می‌شود تا GPU بی‌دلیل روشن نماند

انصاف با مشتری: پروهای ناموفق و پروهایی که از حافظه‌ی نتایج آمده‌اند، **از سهمیه کم نمی‌شوند.** پیام خطا هم صراحتاً می‌گوید «این پرو از سهمیه‌ی شما کم نشد».

جلوگیری از دور زدن سهمیه: اگر اپ دو درخواست هم‌زمان بفرستد، ردیف کاربر در پایگاه داده قفل می‌شود تا هر دو درخواست با هم از بررسی سهمیه رد نشوند (در PostgreSQL فعال است).

حریم خصوصی

سازوکار	توضیح
حذف اطلاعات مکانی	گوشی‌ها موقعیت GPS را داخل فایل عکس ذخیره می‌کنند. همه‌ی اطلاعات EXIF (از جمله مکان) پیش از ذخیره حذف می‌شود. چرخش صحیح عکس قبل از حذف اعمال می‌شود
حذف خودکار عکس	عکس کاربر پس از ۲۴ ساعت خودکار حذف می‌شود (قابل تنظیم). با مقدار `0` ، عکس بلافاصله پس از پایان پرو حذف می‌شود. نتیجه‌ی پرو باقی می‌ماند
حذف با درخواست کاربر	کاربر هر زمان می‌تواند عکسش را از اپ حذف کند و فایل همان لحظه پاک می‌شود
نام فایل غیرقابل حدس	عکس‌ها با نام تصادفی ذخیره می‌شوند؛ نشانی عکس یک نفر قابل حدس زدن نیست
عدم نمایش در پنل مدیریت	کارمندان در پنل، عکس شخصی کاربران را **نمی‌بینند**؛ فقط اطلاعات فنی (ابعاد، تاریخ)
دسترسی فقط به داده‌ی خود	هیچ کاربری نمی‌تواند عکس یا پروی کاربر دیگری را ببیند یا استفاده کند

سازوکار	توضیح
حذف کامل فایل‌ها	با حذف هر رکورد (از هر مسیری: کاربر، پنل، حذف حساب، زمان‌بند)، فایلش از دیسک هم پاک می‌شود

این موارد همان «حریم خصوصی به‌عنوان مزیت رقابتی» پروپوزال است و می‌تواند در صفحه‌ی اول اپ اعلام شود.

پایداری

- **تلاش مجدد با فاصله‌ی افزایشی:** خطای موقت → تلاش دوباره پس از ۵، سپس ۱۰ ثانیه (پیش‌فرض ۲ بار).
- **هیچ کاربری بی‌نهایت منتظر نمی‌ماند:** اگر کارگر صف از کار بیفتد، زمان‌بند هر ۵ دقیقه درخواست‌های گیرکرده (بیش از ۱۵ دقیقه) را «ناموفق» علامت می‌زند تا اپ پیام مناسب نشان دهد.
- **خطای پیش‌بینی‌نشده:** حتی اگر در کد باگی باشد، درخواست «ناموفق» می‌شود، نه «در حال پردازش برای همیشه».
- **جلوگیری از پردازش تکراری:** اگر صف یک درخواست را دو بار تحویل دهد، بار دوم نادیده گرفته می‌شود (یک بار هزینه).
- **جزئیات فنی خطا فقط برای مدیر:** کاربر پیام فارسی ساده می‌بیند؛ متن فنی خطا فقط در پنل مدیریت ذخیره می‌شود.

کمد لباس مجازی

`POST /api/v1/tryon/jobs/<id>/save/` یک نتیجه را در کمد ذخیره و `DELETE` همان مسیر آن را خارج می‌کند. `GET /api/v1/tryon/jobs/?saved=true` فهرست کمد را برمی‌گرداند. کاربر می‌تواند پروهای مختلف را کنار هم مقایسه کند.

واترمارک برند (اختیاری)

اگر مسیر یک فایل PNG لوگو در `TRYON_WATERMARK_IMAGE` قرار گیرد، لوگو به‌صورت نیمه‌شفاف گوشه‌ی تصویر نتیجه قرار می‌گیرد. هنگام اشتراک‌گذاری در واتساپ و اینستاگرام، برند دیده می‌شود (موتور رشد ارگانیک پروپوزال).

گزارش در پنل مدیریت

بالای فهرست درخواست‌های پرو، خلاصه‌ای نمایش داده می‌شود که مستقیماً به هزینه‌ی GPU مربوط است: **تعداد کل، تعداد ناموفق، تعداد پاسخ‌داده‌شده از حافظه (بدون هزینه‌ی GPU)، و میانگین زمان پردازش.**

فهرست endpointها (همه نیازمند ورود؛ مهمان هم کافی است)

متد	مسیر	کاربرد
POST	`/api/v1/tryon/photos/`	بارگذاری و بررسی عکس
GET	`/api/v1/tryon/photos/`	عکس‌های معتبر کاربر
DELETE	`/api/v1/tryon/photos/<id>/`	حذف فوری عکس
POST	`/api/v1/tryon/jobs/`	شروع پرو
GET	`/api/v1/tryon/jobs/<id>/`	پیگیری وضعیت
GET	`/api/v1/tryon/jobs/`	تاریخچه (`?saved=true` برای کمد)

متد	مسیر	کاربرد
POST / DELETE	`/api/v1/tryon/jobs/<id>/save/`	افزودن/حذف از کمد
DELETE	`/api/v1/tryon/jobs/<id>/`	حذف از تاریخچه
GET	`/api/v1/tryon/quota/`	سهمیه‌ی باقی‌مانده‌ی امروز

کدهای خطای ماشین‌خوان (`code`) تا اپ بتواند واکنش مناسب نشان دهد: `photo_not_found` ، `photo_rejected` ، `quota_exceeded` ، `category_not_supported` ، `product_not_supported` ، `photo_expired` .

محدودیت‌های شناخته‌شده (صادقانه)

مورد	توضیح
کفش	پشتیبانی نمی‌شود. محصول کفش با پیام «هنوز فعال نشده» رد می‌شود. منتظر تصمیم کارفرما و تست امکان‌سنجی
آستانه‌ی تشخیص تاری	عدد فعلی (۱۵) محافظه‌کارانه است و با عکس‌های مصنوعی تنظیم شده. **پیش از لانچ باید با ۲۰ تا ۳۰ عکس واقعی مشتری تنظیم شود** تا عکس خوب رد نشود
تشخیص وجود انسان در عکس	سمت سرور بررسی نمی‌شود؛ این کار به کارگر GPU سپرده شده (خروجی `error`). در صورت نیاز، بعداً یک مدل سبک تشخیص بدن اضافه می‌شود
نشانی عکس‌ها	عکس‌ها با نام تصادفی و از طریق وب‌سرور ارائه می‌شوند. برای امنیت بیشتر، در آینده می‌شود از نشانی‌های امضاشده‌ی زمان‌دار (مثل S3) استفاده کرد
قفل سهمیه	در SQLite (محیط توسعه) قفل ردیف کار نمی‌کند؛ در PostgreSQL (سرور اصلی) فعال است

تست‌ها

۳۹ تست خودکار، همه موفق:

- **بررسی عکس (۵):** تبدیل به JPEG، حذف EXIF و اعمال چرخش، کوچک‌سازی، همه‌ی دلایل رد (کوچک، افقی، تاریک، پرنور، تار، فایل غیرتصویری، فرمت نامعتبر، حجم زیاد)، جدا شدن عکس تار از واضح
- **جریان کامل (۱۳):** بارگذاری و حذف عکس با پاک شدن فایل، پرو کامل، جایگاه در صف، پاسخ از حافظه بدون فراخوانی GPU، حفظ فایل مشترک هنگام حذف، محصول بدون پرو، دسته‌ی پشتیبانی‌نشده (کفش)، عکس دیگران، عکس منقضی، کمد مجازی، خصوصی بودن پروها، انتقال داده‌ی مهمان
- **سهمیه (۳):** سقف مهمان با پیام تشویق به ثبت‌نام، سقف کاربر، کم نشدن پروی ناموفق
- **پایداری (۹):** تلاش مجدد موفق، پایان تلاش‌ها با پیام مناسب، عدم تلاش مجدد برای ورودی بد، باگ پیش‌بینی‌نشده، حذف فوری عکس با تنظیم صفر، حذف عکس‌های منقضی بدون حذف عکس در حال استفاده، درخواست گیرکرده، تحویل تکراری، واترمارک
- **RunPod (۸):** موفقیت پس از چند بار پیگیری و ثبت زمان cold start، مهلت و لغو، کلید نامعتبر، خطای سرور، خطای اعلام‌شده توسط کارگر، کار FAILED، تنظیمات ناقص، رد کفش بدون هیچ فراخوانی شبکه
- **پنل مدیریت (۱):** باز شدن همه‌ی صفحات مدیریت

> **وضعیت:** ✅ ساخته و تست شد (با موتور آزمایشی و شبیه‌سازی RunPod). اتصال واقعی به GPU پس از ساخت ایمیج مدل.

بخش ۵ – سبد خرید، سفارش و پرداخت (`apps/orders`)

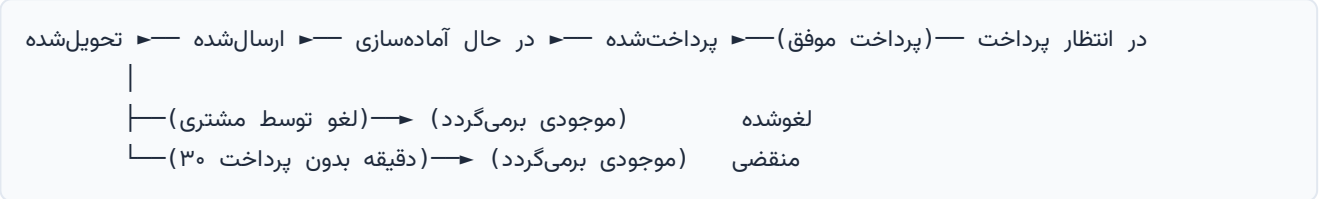
هدف: مسیر کامل خرید، از «افزودن به سبد» تا «پرداخت موفق»، بدون هیچ امکانی برای فروش کالای ناموجود، دوبار پرداخت، یا گم شدن یک سفارش پرداخت‌شده.

مدل‌های داده

مدل	نقش	نکته‌ی مهم
`CartItem`	آیتم سبد خرید	هر سایز/رنگ یک بار؛ افزودن دوباره فقط تعداد را زیاد می‌کند
`Address`	نشانی‌های کاربر	فقط یک نشانی «پیش‌فرض»؛ اولین نشانی خودکار پیش‌فرض می‌شود
`Order`	سفارش	شماره‌ی خوانا (مثل `SK48213907`)، **کپی ثابت نشانی و مبالغ**
`OrderItem`	قلم سفارش	**کپی ثابت نام و قیمت کالا در لحظه‌ی خرید**
`Payment`	هر تلاش پرداخت	شناسه‌ی درگاه، کد پیگیری بانکی، پاسخ خام درگاه

چرا کپی ثابت؟ اگر فردا قیمت یک کالا عوض شود، نامش تغییر کند، یا مشتری نشانی‌اش را ویرایش کند، فاکتور سفارش‌های قبلی نباید تغییر کند. این برای حسابداری و پیگیری شکایت ضروری است.

چرخه‌ی وضعیت سفارش



وضعیت‌های «در حال آماده‌سازی»، «ارسال‌شده» و «تحویل‌شده» را کارفرما از پنل مدیریت تغییر می‌دهد و کد رهگیری مرسوله را وارد می‌کند.

رزرو موجودی – جلوگیری از فروش بیش از موجودی

مشکل کلاسیک فروشگاه‌ها: دو نفر هم‌زمان آخرین عدد یک کالا را می‌خرند. راه‌حل در این پروژه:

1. **هنگام ثبت سفارش** (نه هنگام پرداخت) موجودی کم می‌شود؛ یعنی کالا برای مشتری رزرو می‌شود.
2. کم کردن موجودی با یک دستور شرطی پایگاه داده انجام می‌شود: «فقط اگر موجودی کافی است، کم کن». اگر دو نفر هم‌زمان اقدام کنند، فقط یکی موفق می‌شود؛ این در سطح پایگاه داده تضمین شده، نه در کد.

3. اگر حتی یک قلم از سفارش موجودی نداشته باشد، **کل سفارش لغو می‌شود** (هیچ موجودی‌ای کم نمی‌شود و سبد خرید مشتری دست‌نخورده می‌ماند). نیمه‌کاره ماندن ممکن نیست.
4. اگر مشتری ظرف ۳۰ دقیقه (قابل تنظیم) پرداخت نکند، سفارش خودکار منقضی و موجودی آزاد می‌شود (زمان‌بند هر ۱۰ دقیقه بررسی می‌کند).

مسیر پرداخت

```
۱. POST /api/v1/orders/checkout/          { "address_id": 5, "note": "" }
   ← سفارش با وضعیت «در انتظار پرداخت»

۲. POST /api/v1/orders/<number>/pay/
   ← { "payment_url": "https://payment.zarinpal.com/pg/StartPay/A0000..." }
   اپ این نشانی را در مرورگر گوشی باز می‌کند.

۳. مشتری در درگاه بانک پرداخت می‌کند؛ بانک مرورگر را به سرور برمی‌گرداند:
   GET /api/v1/orders/payment/callback/?Authority=...&Status=OK

۴. سرور مستقیماً با درگاه تأیید (verify) می‌کند، سفارش را «پرداخت‌شده» می‌کند و مرورگر را به اپ برمی‌گرداند:
   sarkhiaboon://payment/result?order=SK48213907&status=success
```

آدرس بازگشت به اپ (`sarkhiaboon://...`) یک «دیپ‌لینک» است که اپ React Native آن را ثبت می‌کند تا پس از پرداخت، کاربر خودکار به صفحه‌ی نتیجه در اپ برگردد. از `.env` قابل تغییر است (`PAYMENT_RESULT_URL`).

امنیت پرداخت

سازوکار	توضیح
مبلغ فقط از سرور	هنگام تأیید با درگاه، مبلغ از رکورد ذخیره‌شده‌ی خود ما خوانده می‌شود، هرگز از نشانی بازگشت. دستکاری نشانی بی‌اثر است
تأیید سرور به سرور	صرف بازگشت با `Status=OK` کافی نیست؛ سرور مستقیماً از درگاه تأیید می‌گیرد
جلوگیری از پردازش دوباره	اگر مشتری صفحه‌ی بازگشت را رفرش کند یا بانک دو بار برگرداند، سفارش فقط یک بار پردازش می‌شود و درگاه دوباره فراخوانی نمی‌شود
پرداخت ناموفق	سفارش «در انتظار پرداخت» می‌ماند و مشتری می‌تواند دوباره تلاش کند (تا پایان مهلت)
قطعی درگاه	اگر درگاه در دسترس نباشد، پیام «اتصال به درگاه برقرار نشد» (کد ۵۰۳)؛ اگر هنگام تأیید قطع باشد، پرداخت در حالت قابل‌پیگیری می‌ماند

حالت خاص: پرداخت پس از پایان مهلت

ممکن است مشتری در صفحه‌ی بانک بماند، مهلت ۳۰ دقیقه تمام شود، سفارش منقضی و موجودی آزاد شود، و بعد پرداخت موفق کند. **پول مشتری هرگز گم نمی‌شود:**

- سفارش به «پرداخت‌شده» تغییر می‌کند.

- اگر کالا هنوز موجود باشد، دوباره رزرو می‌شود.
- اگر در این فاصله کالا به نفر دیگری فروخته شده باشد، در توضیحات سفارش یادداشت «موجودی کافی نیست — تماس با مشتری یا استرداد وجه» ثبت می‌شود تا کارفرما پیگیری کند. موجودی هیچ‌وقت منفی نمی‌شود.

درگاه‌های پرداخت (قابل تعویض)

مقدار `PAYMENT_GATEWAY`	کاربرد
`apps.orders.gateways.SandboxGateway`	توسعه: یک صفحه‌ی «بانک آزمایشی» روی سرور خودمان با دو دکمه‌ی «پرداخت موفق/ناموفق»؛ هیچ پولی جابه‌جا نمی‌شود
`apps.orders.gateways.ZarinpalGateway`	عملیاتی: زرین‌پال (API نسخه‌ی ۴). با `ZARINPAL_SANDBOX=true` روی محیط آزمایشی زرین‌پال

واحد پول: همه‌ی مبالغ داخل پروژه تومان است. زرین‌پال ریال می‌خواهد؛ تبدیل (×۱۰) فقط در یک نقطه و داخل کلاس درگاه انجام می‌شود. افزودن درگاه دیگر (مثلاً IDPay یا سامان) فقط یک کلاس جدید است.

صفحه‌ی بانک آزمایشی وقتی درگاه واقعی فعال باشد، کاملاً غیرفعال است (۴۰۴).

ارسال

هزینه‌ی ارسال ثابت (`ORDER_SHIPPING_COST`) و ارسال رایگان بالاتر از یک مبلغ (`ORDER_FREE_SHIPPING_OVER`)، هر دو به تومان و از `.env`. پیش‌فرض هر دو صفر است تا کارفرما سیاست ارسالش را تعیین کند. اگر بعداً هزینه‌ی ارسال بر اساس شهر لازم شد، به قابلیت «شهر و منطقه» (مرحله‌ی دوم نقشه‌ی راه) مربوط می‌شود.

مهمان‌ها

- مهمان می‌تواند سبد خرید داشته باشد، اما **برای ثبت سفارش باید شماره موبایل تأیید کند** (پاسخ `403` با کد `login_required`؛ اپ صفحه‌ی ورود را نشان می‌دهد).
- هنگام ورود، سبد مهمان با سبد حساب ادغام می‌شود؛ اگر کالایی در هر دو بود، تعدادها جمع می‌شوند (حداکثر تا سقف موجودی).

یکسان‌سازی ورودی نشانی

شماره‌ی گیرنده با همان منطق بخش ۲ یکسان می‌شود (`09...` → `+98...`). کد پستی با ارقام فارسی، خط تیره یا فاصله پذیرفته و به ۱۰ رقم انگلیسی تبدیل می‌شود. **(باگی که تست‌ها پیدا کردند:** در نسخه‌ی اول، محدودیت طول فیلد پیش از یکسان‌سازی اجرا می‌شد و ورودی‌هایی مثل `+989121112233` یا `۱۱۱۱۱-۲۲۲۲۲` رد می‌شدند؛ اصلاح شد.)

پنل مدیریت

- فهرست سفارش‌ها با جست‌وجو (شماره سفارش، نام و موبایل گیرنده، کد رهگیری) و فیلتر وضعیت و استان
- صفحه‌ی سفارش: اقلام و تاریخچه‌ی پرداخت‌ها (فقط‌خواندنی)، تغییر وضعیت، ورود کد رهگیری
- سفارش از پنل قابل ساخت نیست و پرداخت‌ها قابل ویرایش نیستند (جلوگیری از دستکاری سوابق مالی)

فهرست endpointها (همه نیازمند ورود، به‌جز بازگشت از بانک)

متد	مسیر	کاربرد
GET / DELETE	`/api/v1/orders/cart/`	مشاهده / خالی کردن سبد (با جمع، هزینه‌ی ارسال، مبلغ کل)
POST	`/api/v1/orders/cart/items/`	افزودن `{variant_id, quantity}`
PATCH / DELETE	`/api/v1/orders/cart/items/<id>/`	تغییر تعداد / حذف
GET / POST / PATCH / DELETE	`/api/v1/orders/addresses/`	مدیریت نشانی‌ها
POST	`/api/v1/orders/checkout/`	ثبت سفارش از سبد
GET	`/api/v1/orders/` و `/api/v1/orders/<number>/`	فهرست و جزئیات سفارش‌ها
POST	`/api/v1/orders/<number>/pay/`	دریافت نشانی درگاه
POST	`/api/v1/orders/<number>/cancel/`	لغو سفارش پرداخت‌نشده
GET	`/api/v1/orders/payment/callback/`	بازگشت از بانک (عمومی)

کدهای خطای ماشین‌خوان: `empty_cart`، `unavailable`، `max_quantity`، `insufficient_stock`، `gateway_error`، `expired`، `not_cancellable`، `not_payable`، `address_not_found`، `login_required`.

تست‌ها

۳۵ تست خودکار، همه موفق:

- **سبد (۶):** افزودن و ادغام تعداد، ویرایش و حذف، سقف موجودی و سقف تعداد، کالای غیرفعال، هزینه‌ی ارسال و ارسال رایگان، خصوصی بودن سبد، ادغام سبد مهمان
- **نشانی (۳):** یکسان‌سازی شماره و کد پستی فارسی، یک پیش‌فرض، پیش‌فرض شدن اولین نشانی، کد پستی نامعتبر
- **ثبت سفارش (۹):** رزرو موجودی و کپی ثابت قیمت، سبد خالی، الزام ثبت‌نام مهمان، نشانی دیگران، **رقابت بر سر آخرین کالا و لغو کامل سفارش**، جلوگیری از فروش بیش از موجودی در سطح پایگاه داده، لغو و بازگشت موجودی، انقضای خودکار، خصوصی بودن سفارش‌ها
- **پرداخت (۱۰):** رفت‌وبرگشت کامل تا دیپ‌لینک اپ، پرداخت ناموفق و تلاش دوباره، رفرش صفحه‌ی بازگشت، شناسه‌ی جعلی، جلوگیری از پرداخت یا لغو دوباره، سفارش منقضی، پرداخت دیرهنگام با موجودی، پرداخت دیرهنگام بدون موجودی (هشدار و عدم منفی شدن)، قطعی درگاه، غیرفعال بودن بانک آزمایشی در حالت واقعی
- **زرین‌پال (۶):** تبدیل تومان به ریال و ساخت نشانی پرداخت، رد درخواست، استفاده از مبلغ ذخیره‌شده (نه مبلغ نشانی)، کد ۱۰۱، انصراف مشتری بدون فراخوانی درگاه، شناسه‌ی پذیرنده‌ی خالی
- **پنل مدیریت (۱):** باز شدن صفحات سفارش، پرداخت، نشانی و سبد

وضعیت: ✅ ساخته و تست شد. اتصال زرین‌پال با پاسخ‌های شبیه‌سازی‌شده تست شده؛ پیش از راه‌اندازی باید با شناسه‌ی پذیرنده‌ی واقعی کارفرما در محیط آزمایشی زرین‌پال امتحان و نشانی‌های API با مستندات روز زرین‌پال تطبیق داده شود.

بخش ۶ — تست‌ها و بررسی نهایی سرور

هدف: اطمینان از اینکه همه‌ی بخش‌ها نه‌فقط جداجدا، بلکه **با هم و روی یک سرور واقعی** درست کار می‌کنند.

۱. تست‌های خودکار — ۱۱۴ تست، همه موفق

بخش	تعداد
احراز هویت	۱۷
کاتالوگ	۲۳
موتور پرو مجازی	۳۹
سفارش و پرداخت	۳۵
جمع	**۱۱۴**

```
cd backend && DJANGO_DEBUG=true python manage.py test
```

تست‌ها بدون هیچ سرویس خارجی اجرا می‌شوند: پیامک، RunPod، GPU، زرین‌پال و صف پردازش همگی شبیه‌سازی می‌شوند. هیچ تستی پول خرج نمی‌کند یا به اینترنت وصل نمی‌شود. فایل‌های تصویری تست در پوشه‌ی موقت ساخته و پس از پایان پاک می‌شوند.

۲. بررسی زنده‌ی سرتاسری — ۲۴ بررسی، همه موفق

یک سرور واقعی با پایگاه داده‌ی خالی راه‌اندازی شد، داده‌ی نمونه بارگذاری شد، و یک اسکریپت دقیقاً مثل اپ موبایل، از طریق HTTP واقعی کل مسیر یک مشتری را طی کرد:

صفحه‌ی اصلی ← ورود مهمان ← بارگذاری عکس ← رد عکس تاریک با پیام فارسی ← پرو مجازی ← دانلود نتیجه ← پاسخ فوری از حافظه برای پروی تکراری ← سهمیه ← رد کفش ← سبد خرید مهمان ← درخواست کد پیامکی ← ورود و ارتقای حساب مهمان (پروها و سبد حفظ شدند) ← ثبت نشانی با ارقام فارسی ← ثبت سفارش و رزرو موجودی ← صفحه‌ی بانک آزمایشی ← پرداخت ← بازگشت به اپ با دیپ‌لینک ← سفارش پرداخت‌شده ← مستندات API ← پنل مدیریت

این اسکریپت در پروژه نگه داشته شده تا پس از استقرار روی سرور اصلی هم اجرا شود:

```
        python scripts/smoke_test.py --base https://staging.example.ir --log
/path/to/server.log
```

باگی که بررسی زنده پیدا کرد (و تست‌های واحد نه)

در ردیف «محصولات قابل پرو» صفحه‌ی اصلی، **کفش** هم نمایش داده می‌شد؛ در حالی که موتور فعلی کفش را پشتیبانی نمی‌کند. یعنی مشتری روی «پرو کن» می‌زد و خطا می‌گرفت. علت: تشخیص «قابل پرو بودن» فقط تنظیمات محصول را می‌دید، نه توانایی موتور فعال را.

اصلاح: حالا یک منبع واحد (`available_tryon_types`) تعیین می‌کند چه نوع‌هایی واقعاً قابل پرو هستند و کاتالوگ، فیلترها و صفحه‌ی اصلی همه از آن می‌پرسند. وقتی موتوری با پشتیبانی کفش اضافه شود، کفش‌ها **خودکار** قابل پرو می‌شوند، بدون تغییر در کاتالوگ. یک تست دائمی هم اضافه شد تا این باگ برنگردد.

۳. مستندات API — بدون هیچ هشدار

مستندات تعاملی (`/api/docs/`) از خود کد ساخته می‌شود و **۲۸ مسیر** را پوشش می‌دهد. در بررسی اول ۳۵ هشدار داشت (نوع برخی فیلدها «متن» نمایش داده می‌شد در حالی که عدد یا بولین بودند، و الزام ورود بعضی مسیرها دیده نمی‌شد). همه برطرف شد؛ این مستندات **قرارداد رسمی بین سرور و اپ موبایل** است و باید دقیق باشد.

۴. بررسی امنیتی حالت عملیاتی

`manage.py check --deploy` اجرا شد. دو مورد باقی‌مانده عمداً قابل‌تنظیم گذاشته شده‌اند، نه فعال به‌صورت پیش‌فرض:

مورد	دلیل
هدایت خودکار به HTTPS	معمولاً روی nginx انجام می‌شود؛ در صورت نیاز: `DJANGO_SECURE_SSL_REDIRECT=true`
HSTS	پس از فعال شدن، مرورگرها آن را ذخیره می‌کنند و برگرداندنش سخت است. فقط پس از اطمینان از کارکرد HTTPS روی دامنه‌ی نهایی: `DJANGO_HSTS_SECONDS=31536000`

۵. داده‌ی نمونه

```
        python manage.py seed_demo          # دسته، ۱۰ محصول با سایز و موجودی، ۲ بنر، ۳ آیکون ۶ تبلیغاتی
        python manage.py seed_demo --reset  # حذف و ساخت دوباره
```

همه‌ی داده‌های نمونه با برچسب «[نمونه]» در نامشان مشخص‌اند و تصاویرشان اشکال ساده‌ی ساختگی است، تا با کالای واقعی اشتباه نشوند. برای نمایش به کارفرما و برای کار توسعه‌دهنده‌ی اپ پیش از آماده شدن کاتالوگ واقعی.

وضعیت: ✅ انجام شد.

راه‌اندازی و اجرا

اجرای محلی (بدون Docker، برای توسعه)

```
cd backend
pip install -r requirements.txt
export DJANGO_DEBUG=true CELERY_TASK_ALWAYS_EAGER=true
python manage.py migrate
python manage.py seed_demo
python manage.py createsuperuser        # شماره موبایل و رمز مدیر پنل
python manage.py runserver
```

- `CELERY_TASK_ALWAYS_EAGER=true` یعنی پرو مجازی بدون Redis و بدون کارگر جداگانه، همان‌جا اجرا می‌شود (فقط برای توسعه).
- کد ورود پیامکی در خروجی سرور چاپ می‌شود.
- مستندات API: `http://127.0.0.1:8000/api/docs/` — پنل مدیریت: `http://127.0.0.1:8000/admin/`

اجرای کامل با Docker (سرور آزمایشی/اصلی)

```
cp .env.example .env      # مقادیر را ویرایش کنید
docker compose up --build -d
docker compose exec api python manage.py createsuperuser
```

تغییرات لازم برای حالت عملیاتی (در `.env`)

تنظیم	مقدار عملیاتی
`DJANGO_DEBUG`	`false`
`DJANGO_SECRET_KEY`	یک رشته‌ی تصادفی طولانی (حداقل ۵۰ نویسه)
`SMS_BACKEND`	`apps.accounts.sms.KavenegarSMSBackend` + کلید
`TRYON_PROVIDER`	`apps.tryon.providers.runpod.RunPodProvider` + شناسه و کلید RunPod
`PAYMENT_GATEWAY`	`apps.orders.gateways.ZarinpalGateway` + شناسه‌ی پذیرنده، `ZARINPAL_SANDBOX=false`

تنظیم	مقدار عملیاتی
PUBLIC_BASE_URL	نشانی عمومی API، مثلاً https://api.sarkhiaboon.ir

کارهای باقی‌مانده پیش از انتشار

مورد	مسئول	توضیح
اپلیکیشن موبایل React Native	توسعه‌دهنده	مرحله‌ی بعدی ساخت؛ همه‌ی API لازم آماده و مستند است
ایمیج GPU برای RunPod	توسعه‌دهنده	Docker مدل CatVTON با قرارداد بخش ۴؛ بخش «کار اضافه»ی مسیر دوم پروپوزال فنی
تصمیم مسیر (سرویس آماده یا RunPod)	کارفرما	کد برای هر دو آماده است
راهکار پرداخت به RunPod	کارفرما	کارت بین‌المللی لازم است
کفش در نسخه‌ی اول یا نه	کارفرما	اگر بله: تست امکان‌سنجی جداگانه
شهر و منطقه (گزینه‌ی الف یا ب)	کارفرما	ساختار آماده‌ی افزودن است
تنظیم آستانه‌ی تشخیص تاری	توسعه‌دهنده + کارفرما	با ۲۰ تا ۳۰ عکس واقعی مشتری
حساب کاوه‌نگار و قالب پیامک	کارفرما	برای ارسال کد ورود
شناسه‌ی پذیرنده‌ی زرین‌پال	کارفرما	و یک تست در محیط آزمایشی زرین‌پال
تصاویر و اطلاعات واقعی محصولات	کارفرما	به‌ویژه **عکس تمیز لباس برای پرو** (بدون مدل، پس‌زمینه‌ی ساده)
نوتیفیکیشن پوش	توسعه‌دهنده	همراه با ساخت اپ؛ با توجه به محدودیت سرویس‌های گوگل در ایران، سرویس داخلی (مثل پوشه یا نجوا) بررسی شود